



Saad Mughal | F2023-009
Khadijah Zahoor | F2023-956
Sara Haider | F2023-744
Syed Shahmeer Bakht | F2023-339
Total Marks: 10
Due 12 May 2026

Digital Finance & Blockchain Technologies

First Project Report

Web3 To-Do Contract

Course Lecturer: Shamweel Qamar
Section: B

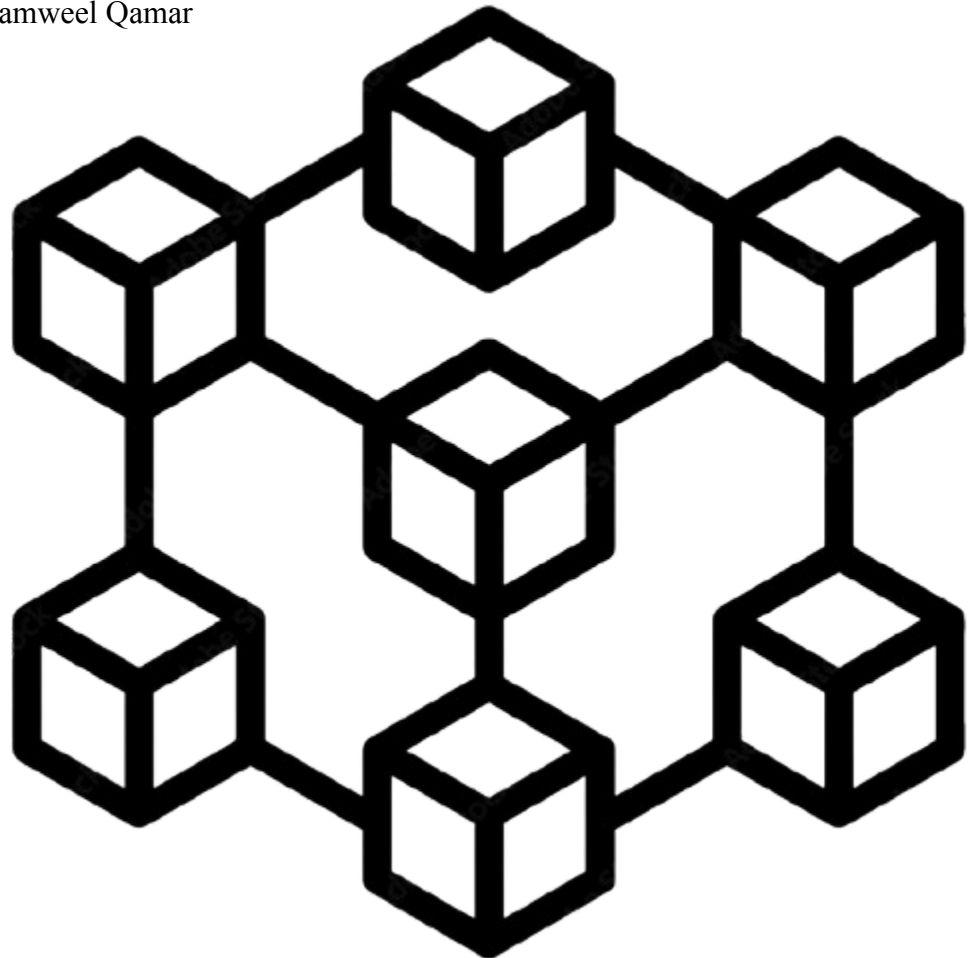


Table of Contents

1. Introduction.....	2
2. Project Description.....	2
3. Proposed Smart Contract Design.....	2
3.1 Data Structures.....	2
3.2 CRUD Operations.....	3
3.3 Code Snippets.....	4
4. Technical Rationale.....	5
5. Deployment Strategy.....	6
5.1 Tools and Network.....	6
5.2 Step-by-Step Deployment Process.....	7
6. Conclusion.....	8

First Project Report: Web3 To-Do Contract

A Decentralized Task Management System on Ethereum Sepolia Testnet

1. Introduction

Traditional to-do applications store user data on centralized servers, requiring users to trust that their data stays secure and unchanged. This project takes a different approach by building a task management system on the blockchain. Using Solidity and the Ethereum Sepolia testnet, all task operations are handled through smart contracts instead of a central database. This report covers the contract design, deployment process, and implementation plan for the DApp.

2. Project Description

The DApp allows any Ethereum wallet holder to create personal tasks, mark them as complete, and delete them when no longer needed. Each task is linked to the wallet address that created it, so no one else can modify or remove it. There is no login system, no password, and no central authority. The blockchain handles identity through wallet signatures automatically.

The intended users are individuals who want full ownership over their productivity data. The logic is transparent and verifiable by anyone on Sepolia Etherscan, meaning the system does not require trust in any company or platform.

At its core, this project is about data ownership. When tasks live on a smart contract, they cannot be altered or deleted by a third party. They exist as long as the blockchain exists.

3. Proposed Smart Contract Design

3.1 Data Structures

The contract (TodoContract.sol) organizes data around two core Solidity constructs.

Construct	Purpose
struct Task	Bundles two fields: a string named content (the task text) and a bool named isCompleted (the completion flag). Grouping related data into a struct keeps storage organized and gas costs predictable.

mapping(uint256 => Task)	Maps each unique task ID (an auto-incremented uint256 key) to its Task struct. Mappings provide O(1) lookup by ID, which is far cheaper on-chain than iterating an array.
mapping(address => uint256[])	Maps each wallet address to the list of task IDs that address has created. This allows the contract to retrieve only the tasks that belong to the caller.
uint256 taskCount	A global counter incremented on every createTask call. It serves as the next available ID and lets the frontend know the total number of tasks without scanning storage.

3.2 CRUD Operations

Every interaction with the contract maps to one of the four CRUD categories. The table below summarises each function before the code snippets illustrate the two most important ones.

CRUD	Function	Visibility	What It Does
Create	createTask(string)	<i>public</i>	Accepts a task string, increments taskCount, stores a new Task struct, and pushes the new ID into the caller's address mapping.
Read	getTask(uint256)	<i>public view</i>	Returns the content string and isCompleted bool for a given ID. View functions cost no gas when called off-chain.
Read	getMyTasks()	<i>public view</i>	Returns the full array of task IDs belonging to msg.sender, allowing the frontend to batch-fetch the caller's tasks.
Update	toggleTask(uint256)	<i>public</i>	Flips isCompleted from false to true (or back). Verifies via require that only the original creator can toggle their own task.
Update	editTask(uint256, string)	<i>public</i>	Replaces the content string of an existing task. Ownership is verified before the write, and an event is emitted.
Delete	deleteTask(uint256)	<i>public</i>	Resets the Task struct to default values (empty string, false). On-chain data cannot be erased, so resetting to defaults is the gas-efficient equivalent of deletion.

3.3 Code Snippets

The two snippets below represent the core of the contract. They demonstrate the data structure and the Create operation, which is the foundation every other function builds on.

Task Struct and State Variables

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.19;

contract TodoContract {

    // ---- Data structures ----

    struct Task {

        string content;           // The task description

        bool isCompleted; // false = pending, true = done

        address owner;           // Wallet that created this task

    }

    uint256 public taskCount;           // Total tasks ever created

    mapping(uint256 => Task) public tasks;           // ID => Task

    mapping(address => uint256[]) private ownerTasks; // Addr => [IDs]

    // ---- Events ----

    event TaskCreated(uint256 indexed id, address indexed owner, string content);

    event TaskToggled(uint256 indexed id, bool isCompleted);

    event TaskDeleted(uint256 indexed id);
```

createTask Function (Create)

```
function createTask(string memory _content) public {

    require(bytes(_content).length > 0, "Task cannot be empty");

    require(bytes(_content).length <= 280, "Content too long");


    taskCount++;                // Increment before use (IDs start at 1)

    tasks[taskCount] = Task({

        content:        _content,

        isCompleted: false,

        owner:          msg.sender // EVM provides caller identity automatically

    });

    ownerTasks[msg.sender].push(taskCount);

    emit TaskCreated(taskCount, msg.sender, _content);

}
```

Note on Delete: Blockchain storage is immutable. deleteTask resets the Task struct to Solidity defaults (empty string, false, address(0)) rather than erasing it. This is intentional and technically correct: resetting costs less gas than truly removing a mapping entry, and the deleted state remains publicly auditable on the chain.

4. Technical Rationale

Property	Why It Matters for a To-Do Contract
----------	-------------------------------------

Immutability	Once a task is saved on the contract, it gets recorded on the blockchain and timestamped in a block. After that, it cannot be changed by anyone, including an admin. This means the task history stays permanent and transparent, creating a reliable record that can't be tampered with.
Data Ownership	The user's wallet address is the only key that can modify their own tasks. There is no central server that can be shut down, hacked, or sold to a third party, so the user owns their data unconditionally.
Transparency	The contract bytecode and all historical transactions are publicly visible on Sepolia Etherscan. Anyone can independently verify that taskCount is incrementing correctly and that ownership checks are enforced exactly as the code describes.
Gas Efficiency	uint256 is the EVM's native word size, so it costs less gas than smaller types like uint8. String length is capped at 280 bytes (a tweet-length limit) via a require guard, preventing unexpectedly large storage writes that would spike transaction fees.
Access Control	Every state-changing function that targets an existing task contains require(tasks[id].owner == msg.sender), ensuring that one user can never complete, edit, or delete another user's tasks, without any off-chain authentication layer.

5. Deployment Strategy

5.1 Tools and Network

Category	Tool	Role in Deployment
Development Environment	Remix IDE	Browser-based Solidity editor and compiler. Produces the contract Bytecode (machine code the EVM runs) and ABI (the JSON interface the frontend uses to call functions). Zero installation required.
Wallet Provider	MetaMask	Browser extension that holds the deployer's private key, signs the deployment transaction, and pays the gas fee on the testnet. Acts as the bridge between Remix and the blockchain.

Test Network	Sepolia Testnet	An Ethereum testnet that mirrors Mainnet behaviour exactly. Gas fees are paid in test ETH obtained free from faucets, so there is zero financial risk during development or grading.
Test ETH Source	Sepolia Faucet	sepoliafaucet.com and the Alchemy Sepolia faucet both dispense free test ETH. A free account is all that is required; no credit card or real cryptocurrency is needed.
Verification	Sepolia Etherscan	After deployment, the contract source code is submitted to Etherscan for public verification. This lets anyone read the ABI and call contract functions directly through the Etherscan web interface.
Frontend Integration	React + ethers.js	ethers.js v6 connects the React frontend to the deployed contract using its address and ABI. Users interact with a familiar web UI while all data operations happen on-chain.
Hosting	GitHub Pages / Vercel	Both platforms offer free static hosting. The compiled React app is pushed to GitHub and deployed in one click, making the DApp publicly accessible at no cost.

5.2 Step-by-Step Deployment Process

1. **Draft:** The first step is to write the TodoContract.sol file in Remix IDE. This includes defining the struct, mappings, CRUD functions, access controls, and events. Each function is kept simple and focused, with comments added to make the code easier to understand.
2. **Compile:** Next, the Solidity compiler version 0.8.19 is selected in Remix. Once the code compiles successfully, it generates the Bytecode (which is deployed to the blockchain) and the ABI file (which is later used to connect the frontend with the smart contract).
3. **Test:** Before deploying, the contract is tested using Remix's built-in JavaScript VM. This helps simulate different scenarios locally. For example, trying to update a task as a non-owner should fail, empty task inputs should be rejected, and deleting a task should reset its values instead of breaking the contract.
4. **Deploy:** After testing, the environment is switched to "Injected Provider" in Remix, which connects to MetaMask. MetaMask is set to the Sepolia testnet and funded with test ETH. The deployment is then confirmed through MetaMask, where gas fees are shown before final approval.

5. **Verify:** Once deployed, the contract address is copied and opened in Sepolia Etherscan. The source code is submitted for verification so the contract becomes publicly readable and its functions can be directly interacted with from the explorer.
6. **Integrate:** Finally, the contract address and ABI are added into the React frontend. Using ethers.js, the frontend interacts with the contract functions like creating, reading, updating, and deleting tasks. The full user flow is then tested in the browser to ensure everything works smoothly.

4.3 Full Stack Summary

- Smart Contract Language: Solidity 0.8.19
- IDE: Remix IDE (browser-based, free)
- Wallet: MetaMask browser extension (free)
- Testnet: Sepolia Ethereum Testnet (free test ETH via faucets)
- Frontend: React.js + ethers.js v6 (open-source)
- Hosting: GitHub Pages or Vercel (free tier)
- Explorer: Sepolia Etherscan (public contract verification, free)

6. Conclusion

The Web3 To-Do Contract shows, in a simple and practical way, how basic blockchain ideas can be used in a real-world application. In a normal web app, features like creating, reading, updating, and deleting data are usually handled through a REST API and a database. In this project, however, all of that logic is written in Solidity and runs on the Ethereum Virtual Machine.

Instead of just being theoretical ideas, concepts like data ownership, immutability, transparency, and gas efficiency are actually built into the design of the contract and reflected in how it works.

The project is being developed entirely using free tools such as Remix IDE, MetaMask, Sepolia testnet faucets, GitHub, and Vercel, so no real money or paid services are needed. Moving forward, the next steps are to finalize and test the smart contract, deploy it on the Sepolia testnet, and connect it with a React frontend to create a fully working decentralized application.